



Efficient spectral Analysis 1

効率的なスペクトル分析1

SILAA, Victor Alex
釧路工業高等専門学校
創造工学科(情報分野)
silaa@kushiro.kosen-ac.jp

離散フーリエ変換 (DFT) の効率的・標準的なアルゴリズムとして、高速フーリエ変換 (FFT) について理解しよう。

Let's understand Fast Fourier Transform (FFT) as an efficient and standard algorithm for Discrete Fourier Transform (DFT).

教科書の該当範囲: 第4章

Textbook Coverage: Chapter 4

素朴な DFT Naive DFT

前回の実習において DFT を数学的定義の通りに実装し、一応、スペクトル分析には成功した。

In the previous exercise, DFT was implemented as defined mathematically, and succeeded in spectral analysis.

計算方法:

Calculation method

$$\mathbf{X}_N = F_N \mathbf{x}_N$$

$$\begin{pmatrix} X_0 \\ X_1 \\ X_2 \\ \vdots \\ X_{N-1} \end{pmatrix} = \begin{pmatrix} W_N^0 & W_N^0 & W_N^0 & \cdots & W_N^0 \\ W_N^0 & W_N^1 & W_N^2 & \cdots & W_N^{N-1} \\ W_N^0 & W_N^2 & W_N^4 & \cdots & W_N^{2(N-1)} \\ \vdots & \vdots & \vdots & & \vdots \\ W_N^0 & W_N^{N-1} & W_N^{2(N-1)} & \cdots & W_N^{(N-1)^2} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{N-1} \end{pmatrix}$$

→ N : 標本数 number of samples

→ F_N : DFTの変換行列 F_N : transformation matrix of DFT

→ x_N : 離散波形のベクトル, vector of discrete waveforms, n : 標本番号(時間) sample number (time)

→ X_N : 離散スペクトルのベクトル vector of discrete spectrum, k : 高調波番号(周波数) Harmonic number(frequency)

$$X_k = \sum_{n=0}^{N-1} x_n W_N^{kn}, \quad W_N^{kn} = \exp(-j 2\pi kn/N)$$

→ 問題点: 計算量 $O(N^2)$ problem: Complexity

→ 主に、行列内の回転子 W_N^{kn} の算出とベクトルとの乗算の影響が大 The main influence is the calculation of the rotator W_N^{kn} in the matrix and its multiplication with the vector.



数学的な準備として，回転子 $W_N = \exp(-j 2\pi/N)$ の性質を理解しておこう。

To prepare mathematically, let us understand the properties of the rotor.

1. xW_N^a : 数 x を複素平面上で時計回りに a/N 回転 number x clockwise a/N rotation on the complex plane

→ これだけ理解しておけば，下記は自明．特に， W_N^a の添字部分が分数 $\frac{a}{N}$ に見えれば吉．

With this understanding, the following is self-explanatory. In particular, if the subscript of W_N^a looks like the fraction $\frac{a}{N}$, it is good.



以下, m は整数, a および b は任意の数, として

Hereinafter, m is an integer, and b are arbitrary numbers

$$2. W_N^{mN} = 1$$

→要するに, 整数 $m \times 360$ 度の回転 → 元に戻り, 変化なし, つまり1倍, ってこと

In short, an integer $m \times 360$ degree rotation → back to the original, no change, i.e., 1x.



3. $W_N^{N/2} = -1$

→ まあ, 半回転だよな

Well, it's a half turn, isn't it?

4. $W_N^{mN+a} = W_N^a$

→ いわゆる, 周期性ってヤツ

So-called periodicity.



$$5. W_N^{a+b} = W_N^a W_N^b$$

→ 回転の回転は回転なのだ. or 指数関数の性質そのままだべさ.

Rotation of rotation is rotation. or The properties of the exponential function are the same.

$$6. W_N^{ab} = W_{N/a}^b$$

→ これは, $ab/N = b/(N/a)$ から来てます

ab/N comes from $b/(N/a)$

DFT の変換行列について，小規模なものを具体的に計算し，その性質を確認しておこう

Let's check the properties of the transformation matrix of the DFT by computing a small-scale version of it.

$N=1$ の場合 →

In the case of $N = 1$

$$F_1 = \begin{pmatrix} W_1^0 \end{pmatrix} = 1$$

$N=2$ の場合 →

In the case of $N = 2$

$$F_2 = \begin{pmatrix} W_2^0 & W_2^0 \\ W_2^0 & W_2^1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$N=3$ の場合 →
In the case of $N = 3$

$$F_3 = \begin{pmatrix} W_3^0 & W_3^0 & W_3^0 \\ W_3^0 & W_3^1 & W_3^2 \\ W_3^0 & W_3^2 & W_3^4 \end{pmatrix} = \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & W_3^1 & W_3^2 \\ \cdot & W_3^2 & \textcolor{red}{W}_3^1 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & \exp(-j2\pi/3) & \exp(-j4\pi/3) \\ 1 & \exp(-j4\pi/3) & \exp(-j2\pi/3) \end{pmatrix}$$

$N=4$ の場合 →
In the case of $N = 4$

$$F_4 = \begin{pmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{pmatrix} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & W_4^1 & \textcolor{red}{-}W_4^0 & \textcolor{red}{-}W_4^1 \\ \cdot & \textcolor{red}{-}W_4^0 & W_4^0 & \textcolor{red}{-}W_4^0 \\ \cdot & \textcolor{red}{-}W_4^1 & \textcolor{red}{-}W_4^0 & \textcolor{red}{W}_4^1 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{pmatrix}$$

変換行列を求めよ, とか試験問題がありそうですね. DFT を手動で実行せよ, とかも

There might be an exam question like "Find the transformation matrix". Or, perform the DFT manually.

DFT の変換行列の特徴

Characteristics of the transformation matrix of the DFT

1. 単純な数値 1 や -1 となる場合が多い

Often a simple number 1 or -1

→ 計算機を働かせる意味がありませんよね

There's no point in working the calculator, is there?



2. 単複数個の等しい数値が周期的に現れる.

A single equal number appears periodically.

→ それらの計算は1回ずつで充分ですよ.

It is enough to do those calculations one at a time, right?

結論: 素朴な DFT の計算には無駄が多い

Conclusion: naive DFT calculations are wasteful.

いよいよ、高速フーリエ変換 (FFT) "fast Fourier transform" を導出してみよう. .

Finally, let's derive the "fast Fourier transform" (FFT).

標本数を $N=2^m$ に限定し, N 次の DFT を $N/2$ 次の DFT によって再帰的に定義します

Limit the number of samples to $N=2^m$ and define the N -order DFT recursively by the $N/2$ -order DFT

教科書の内容とは異なる方法ですが、結果的には同じことになりますよ。比較してください。

It's a different method than in the textbook, but the result is the same. Compare.

$N=2 \rightarrow 1$ の場合

In the case where $N = 2$

$$\begin{aligned} \mathbf{X}_2 &= F_2 \mathbf{x}_2 \\ \begin{pmatrix} X_0 \\ X_1 \end{pmatrix} &= \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} x_0 + x_1 \\ x_0 - x_1 \end{pmatrix} \\ &= \begin{pmatrix} F_1(x_0 + x_1) \\ F_1(x_0 - x_1) \end{pmatrix} \end{aligned}$$

そういえば, $F_1=1$ でしたよね

Remember that $F_1=1$

要するに, 1回の F_2 を2回の F_1 で計算可能ということですね

So, in short, one time F_2 can be computed with two times F_1

$N=4 \rightarrow 2$ の場合

In the case where $N = 4$

$$\mathbf{X}_4 = F_4 \mathbf{x}_4$$
$$\begin{pmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

スペクトルを偶数番と奇数番とに分類，波形を前半と後半とに分割し，整理すると

The spectrum is classified into even and odd numbers, and the waveform is divided into the first half and the second half.

偶数番のスペクトル

Even-numbered spectrum

$$\begin{aligned} \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} &= \begin{pmatrix} 1 & 1 & | & 1 & 1 \\ 1 & -1 & | & 1 & -1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_2 \\ x_3 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x_0 + x_2 \\ x_1 + x_3 \end{pmatrix} \\ &= F_2 \begin{pmatrix} x_0 + x_2 \\ x_1 + x_3 \end{pmatrix} \end{aligned}$$

はい, F_4 の半分が F_2 で表せた. スペクトル x_4 の偶数番部分を算出できます

Yes, half of F_4 could be represented by F_2 . The even-numbered part of the spectrum x_4 can be calculated.

波形 x_4 の前半 + 後半とを改めて, 波形 x_2 とした感じです

The first half of the waveform x_4 plus the second half is like the waveform x_2 again

数式の変形としては, 最初の 2×4 行列の左右半分が等しいことを利用しましたよ

As a variant of the formula, I took the advantage of the fact that left and right halves of the first 2×4 matrix are equal.

$$\begin{aligned} \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} &= \begin{pmatrix} 1 & -j & -1 & j \\ 1 & j & -1 & -j \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x_0 - x_2 \\ -j(x_1 - x_3) \end{pmatrix} \\ &= F_2 \begin{pmatrix} x_0 - x_2 \\ -j(x_1 - x_3) \end{pmatrix} \end{aligned}$$

奇数番のスペクトル

Odd-numbered spectrum

残り半分も F_2 で表せた. スペクトル x_4 の奇数番部分です

Yes, half of F_4 could be represented by F_2 . The even-numbered part of the spectrum x_4 can be calculated.

波形ベクトル x_4 の前半-後半のベクトルを作り, 各要素に回転子を適用し, それを改めて, 波形ベクトル x_2 とした感じです

It is like creating the first-half vector of the wave vector x_4 applying the rotator to each element, and renaming it the wave vector x_2

数式変形では, 2×4 行列の左右半分の類似性などを利用しました

In transforming the formula, we used the similarity between the left and right halves of a 2×4 matrix, etc.

$N=8 \rightarrow 4$ の場合

In the case where $N = 4$

ここは自学自習として導出しよう.

Lets derive this as a self-study

一般化 $N = 2^m \rightarrow 2^{m-1}$

Generalize $N =$

偶数番のスペクトル

Even-numbered spectrum

奇数番のスペクトル

Odd-numbered spectrum

$$\mathbf{X}_N = F_N \mathbf{x}_N$$

$$\begin{pmatrix} X_0 \\ X_2 \\ \vdots \\ X_{N-2} \end{pmatrix} = F_{N/2} \begin{pmatrix} x_0 + x_{N/2} \\ x_1 + x_{N/2+1} \\ \vdots \\ x_{N/2-1} + x_{N-1} \end{pmatrix}$$

$$\begin{pmatrix} X_1 \\ X_3 \\ \vdots \\ X_{N-1} \end{pmatrix} = F_{N/2} \begin{pmatrix} W_N^0(x_0 - x_{N/2}) \\ W_N^1(x_1 - x_{N/2+1}) \\ \vdots \\ W_N^{N/2-1}(x_{N/2-1} - x_{N-1}) \end{pmatrix}$$



再帰的に $N = 2^m \rightarrow 2^{m-1} \dots \rightarrow 1$ と計算規模を縮小し，結果的には1次の DFT (加減算) が N 回だけ実行されることになります

Recursively $N = 2^m \rightarrow 2^{m-1} \dots \rightarrow 1$ and reducing the computational scale, resulting in a first-order DFT (addition/subtraction) being performed only N times

ただし，回転子の計算コストは消えていませんよ．波形ベクトルの分割処理に残されています

However, the computational cost of the rotor has not disappeared. It is left in the wave vector segmentation process.

波形ベクトルには，回転子 W が規則的に並んでいるので，漸化式によって指数関数 $\rightarrow$ 乗算に変形すれば，更なる高速化も期待できますね

Since the wave vector contains a regular sequence of rotors W , we can expect further speedups by transforming the wave vector from an exponential function to a multiplicative function using a reduction formula.

計算量 Computational complexity

1. 再帰のどの段階でも、回転子との乗算の計算回数は $N/2$ 回

At any stage of the recursion, the number of computations of the multiplication with the rotor is $N/2$ times

→ 回転子は、2分割した波形ベクトルの後半部分だけに現れるので

The rotor appears only in the second half of the bisected wave vector, so

→ 行列は消滅します。再帰の最終段階 $F_1 = 1$ となるので

The matrix has vanished. Since the final stage of recursion $F_1 = 1$

2. 再帰による繰り返し回数は $m - 1 = \log_2 N - 1$ 回 The number of iterations by recursion is $m - 1 = \log_2 N - 1$ times

3. したがって、FFT の計算量は $O(N \log N)$ Therefore, the computational complexity of the FFT is $O(N \log N)$

逆変換 (IFFT) : IDFT と同様に、FFT の計算方法を転用可能

Inverse transform (IFFT): Similar to IDFT, FFT calculation method can be converted

→ 回転子 W の指数部の符号を反転し、結果を $1/N$ するだけ Just invert the sign of the exponential part of the rotor W , and the result is $1/N$

FFT では、再帰の過程で計算結果のベクトルをスクランブルしてしまうため、これを解除するための後処理が必要となる

The FFT requires post-processing to unscramble the vector of computed results during the recursion process.

→コスト削減のために必要な追加コスト. いわゆるオーバーヘッドですね

Additional costs needed to reduce costs. So-called overhead

スクランブルの具体例

Specific examples of scrambling

$k_{(2)}$	$N = 16$	$N = 8$	$N = 4$	$N = 2$	$N = 1$	$k'_{(2)}$
0000	X_0	X_0	X_0	X_0	X_0	0000
0001	X_1	X_2	X_4	X_8	X_8	1000
0010	X_2	X_4	X_8	X_4	X_4	0100
0011	X_3	X_6	X_{12}	X_{12}	X_{12}	1100
0100	X_4	X_8	X_2	X_2	X_2	0010
0101	X_5	X_{10}	X_6	X_{10}	X_{10}	1010
0110	X_6	X_{12}	X_{10}	X_6	X_6	0110
0111	X_7	X_{14}	X_{14}	X_{14}	X_{14}	1110
1000	X_8	X_1	X_1	X_1	X_1	0001
1001	X_9	X_3	X_5	X_9	X_9	1001
1010	X_{10}	X_5	X_9	X_5	X_5	0101
1011	X_{11}	X_7	X_{13}	X_{13}	X_{13}	1101
1100	X_{12}	X_9	X_3	X_3	X_3	0011
1101	X_{13}	X_{11}	X_7	X_{11}	X_{11}	1011
1110	X_{14}	X_{13}	X_{11}	X_7	X_7	0111
1111	X_{15}	X_{15}	X_{15}	X_{15}	X_{15}	1111

再帰の各段階でベクトル要素を番号の偶奇性によって分類し、順序変更を繰り返していますよ。
そのため、例えば、 $N=1$ の結果の x_1 として、本来の $N=16$ の結果として期待される x_8 の
値が来てしまいました

At each stage of the recursion, the vector elements are classified according to their even-numbered oddness and reordered repeatedly. So, for example, as the result x_1 of $N=1$ comes the expected value x_8 as a result of the original $N=16$

→要素の順序が完全に破壊されてしまった？ わけでもない Has the order of elements been completely destroyed? Not necessarily

スクランブルの解除方法: ビットリバーサル “bit-reversal” How to unscramble: bit-reverse "bit-reversal"

実装方法a: ビットリバーサルの計算関数を定義しておき、出力時に利用 → $.X[\text{Bitrev}(k)].$

Implementation method a: Define a calculation function for bit reversal and use it for output.

実装方法b: ビットリバーサルの対応表を定義しておき、出力時に利用 → $.X[\text{map}[k]].$

FFT の計算方法を可視化してみる

Visualize how the FFT is calculated

→授業時に図を描きます Draw a diagram during a class

バタフライ演算 "butterfly operation"

$$\begin{pmatrix} x_0 \\ x_1 \end{pmatrix} \rightarrow \begin{pmatrix} x_0 + x_1 \\ x_0 - x_1 \end{pmatrix}$$

要するに、二次の DFT 演算なのですが、これを2入力・2出力のデータフロー図として表すと、蝶が現れます。 In essence, it is a quadratic DFT operation, but if you represent it as a two-input, two-output data flow diagram, a butterfly appears.

バタフライダイアグラム "butterfly diagram"

→多入力の FFT をバタフライ演算(二次の DFT)の組み合わせによって図式化したものです

The multiple-input FFT is schematically represented by a combination of butterfly operations (second-order DFTs)